

**Московский государственный технический
университет им. Н. Э. Баумана**

Факультет «Информатика и системы управления»
Кафедра ИУ5 «Системы обработки информации и
управления»

Курс «Парадигмы и конструкции языков
программирования»
Реферат на тему:
Применение паттернов проектирования в C#

Выполнил:
Студент группы ИУ5-31Б
Блинкова Ксения

Проверил:
Гапанюк Ю. Е.

2025 г.

В современной разработке программного обеспечения особую значимость приобретает не просто написание работающего кода, а создание архитектурно грамотных, гибких и расширяемых систем. Именно эту задачу помогают решать паттерны проектирования — типовые, проверенные временем решения распространенных проблем, возникающих при проектировании программного обеспечения. Паттерны представляют собой не готовый код для копирования, а концептуальные шаблоны, описывающие оптимальные способы организации взаимодействия между классами и объектами. Их ценность заключается в создании общего языка для разработчиков, позволяя одним термином описать сложную архитектурную концепцию. Когда один программист говорит другому "здесь мы используем Фабричный метод", это сразу передает определенный подход к проектированию без необходимости детальных объяснений. Паттерны также способствуют соблюдению ключевых принципов качественного кода, таких как SOLID, уменьшают связанность между компонентами системы и повышают их связность внутри, что в итоге приводит к созданию более читаемого, тестируемого и сопровождаемого программного обеспечения.

Концепция паттернов проектирования была систематизирована и популяризирована в 1994 году знаменитой книгой "Design Patterns: Elements of Reusable Object-Oriented Software" авторов Эриха Гаммы, Ричарда Хелма, Ральфа Джонсона и Джона Влиссидеса, которых стали называть "Бандой четырех". Эта работа описала 23 классических паттерна, разделенных на три основные категории: порождающие, структурные и поведенческие. Порождающие паттерны беспокоятся о гибком создании объектов без внесения в программу лишних зависимостей, к ним относится изучаемый в данной работе Фабричный метод. К поведенческим паттернам, определяющим способы взаимодействия объектов, принадлежит Наблюдатель. Третья категория — структурные паттерны — описывает способы композиции классов и объектов для образования более крупных структур. Выбор именно Фабричного метода и Наблюдателя для детального рассмотрения обусловлен их фундаментальной важностью и широкой распространенностью в реальной разработке. Эти паттерны решают принципиально разные, но часто встречающиеся задачи

Язык программирования C# выбран в качестве платформы для исследования не случайно. Его объектно-ориентированная природа, богатая система типов и четкий синтаксис делают его идеальной средой для изучения и реализации паттернов проектирования. Именно благодаря этим характеристикам C# становится прекрасным вариантом для практического изучения паттернов проектирования. Рассмотрим два из них — Фабричный метод и Наблюдатель — которые, несмотря на принадлежность к разным категориям, прекрасно иллюстрируют философию повторно используемых архитектурных решений.

Паттерн "Фабричный метод"

В процессе разработки программного обеспечения разработчики постоянно сталкиваются с необходимостью создания объектов различных классов. На первый взгляд, задача кажется простой — достаточно использовать оператор `new` и указать нужный класс. Однако в реальных, особенно крупных и сложных системах, такой

прямой подход приводит к серьезным архитектурным проблемам, которые становятся особенно явными при необходимости расширения или модификации системы. Представим классическую ситуацию, характерную для многих бизнес-приложений: изначально разрабатывается система управления автомобильными перевозками, где вся логика работы завязана на классе `Truck`. Код в различных модулях системы создает экземпляры этого класса, вызывает его методы, передает объекты этого типа между компонентами. Система успешно функционирует, бизнес развивается, и в определенный момент возникает потребность в расширении функциональности — добавлении поддержки морских перевозок с использованием судов. На этом этапе разработчики сталкиваются с фундаментальной проблемой: весь существующий код жестко привязан к конкретному классу.

Каждый вызов `new Truck()`, каждое объявление переменной типа `Truck`, каждый метод, принимающий параметр типа `Truck`, становится точкой, требующей модификации. В небольших системах это может быть несколько десятков мест, в крупных корпоративных приложениях — сотни или даже тысячи.

Рассмотрим систему управления перевозками. Изначально она работает только с автомобильными перевозками:

```
public class Truck
{
    public void Deliver() => Console.WriteLine("Доставляю груз по дороге");
}

public class LogisticsSystem
{
    public void ProcessOrder()
    {
        Truck truck = new Truck(); // Жесткая привязка
        truck.Deliver();
    }
}
```

Попытка решить проблему напрямую приводит к появлению условных конструкций:

```
public void ProcessOrder(string transportType)
{
    if (transportType == "truck")
    {
        Truck truck = new Truck();
        truck.Deliver();
    }
    else if (transportType == "ship")
    {
        Ship ship = new Ship();
        ship.Deliver();
    }
}
```

Этот подход нарушает принцип открытости/закрытости. Согласно этому

принципу, программные сущности должны быть открыты для расширения, но закрыты для модификации. В нашем примере с транспортом каждый новый вид перевозок требует модификации существующего кода, что прямо противоречит этому принципу.

Решение через паттерн "Фабричный метод"

1. Создание общего интерфейса

```
public interface ITransport
{
    void Deliver();
}
```

2. Реализация конкретных классов

```
public class Truck : ITransport
{
    public void Deliver() => Console.WriteLine("Грузовик везет груз по дороге");
}

public class Ship : ITransport
{
    public void Deliver() => Console.WriteLine("Корабль плывет по морю");
}
```

3. Базовый класс с фабричным методом

```
public abstract class LogisticsCompany
{
    // Фабричный метод
    public abstract ITransport CreateTransport();

    public void PlanDelivery()
    {
        ITransport transport = CreateTransport(); // Создание через фабричный метод
        transport.Deliver();
    }
}
```

4. Конкретные реализации

```
public class RoadLogistics : LogisticsCompany
{
    public override ITransport CreateTransport()
    {
        return new Truck();
    }
}

public class SeaLogistics : LogisticsCompany
{
    public override ITransport CreateTransport()
```

```

    {
        return new Ship();
    }
}

```

Преимущества подхода:

Соблюдение принципа открытости/закрытости — система расширяется без модификации существующего кода

Слабая связанность — клиентский код зависит от абстракций, а не от конкретных классов

Централизация логики создания — весь код создания объектов в одном месте

Использование:

```

class Program
{
    static void Main()
    {
        LogisticsCompany roadCompany = new RoadLogistics();
        roadCompany.PlanDelivery(); // Использует грузовик

        LogisticsCompany seaCompany = new SeaLogistics();
        seaCompany.PlanDelivery(); // Использует корабль
    }
}

```

В своей основе Фабричный метод — это не про создание объектов, а про управление зависимостями. Паттерн решает фундаментальную проблему: как сделать так, чтобы код, который использует объекты, не зависел от того, как эти объекты создаются. В конечном счете, паттерн учит важнейшему принципу проектирования: система должна быть построена так, чтобы её можно было изменять, не ломая то, что уже работает. Это и есть суть профессионального подхода к созданию программного обеспечения, которое живет и развивается вместе с бизнесом, для которого оно создано.

Паттерн "Наблюдатель"

В современной разработке программного обеспечения часто возникает необходимость в организации эффективной коммуникации между различными компонентами системы. Особенно актуальной эта задача становится в ситуациях, когда изменение состояния одного объекта должно автоматически приводить к реакции множества других объектов. Классическим примером может служить система подписки на новости, где редакция публикует материалы, а читатели получают уведомления о новых публикациях.

Например: новостной портал публикует статьи, и различные клиенты системы должны получать информацию об этих публикациях. Без использования специализированных архитектурных решений возникает проблема либо постоянного опроса сервера со стороны клиентов, что приводит к неэффективному использованию

ресурсов, либо массовой рассылки уведомлений всем пользователям, включая тех, кто не интересуется конкретной темой.

Паттерн "Наблюдатель" предлагает хорошее решение этой проблемы через реализацию механизма подписки. Суть паттерна заключается в создании слабосвязанной системы, где объекты, генерирующие события (издатели), и объекты, реагирующие на эти события (наблюдатели), взаимодействуют через четко определенный интерфейс, не зная конкретных деталей реализации друг друга.

Рассмотрим практическую реализацию паттерна "Наблюдатель" на языке C# для системы новостного портала. Первым шагом является определение интерфейсов, которые зададут общий контракт для всех участников системы взаимодействия.

```
// Интерфейс для всех наблюдателей (подписчиков)
public interface INewsObserver
{
    void Update(string newsTitle, string newsContent);
}

// Интерфейс для издателя (новостного портала)
public interface INewsPublisher
{
    void Subscribe(INewsObserver observer);
    void Unsubscribe(INewsObserver observer);
    void NotifyObservers(string newsTitle, string newsContent);
}
```

Эти интерфейсы создают основу для взаимодействия: INewsObserver определяет метод Update, который будет вызываться при получении уведомления, а INewsPublisher предоставляет методы управления подписками и уведомления подписчиков.

Следующим этапом является реализация конкретного издателя — новостного портала, который будет управлять подписчиками и публиковать новости.

```
// Конкретная реализация издателя - новостной портал
public class NewsPortal : INewsPublisher
{
    private List<INewsObserver> _subscribers = new List<INewsObserver>();
    private string _portalName;

    public NewsPortal(string name)
    {
        _portalName = name;
        Console.WriteLine($"Создан новостной портал: {_portalName}");
    }

    // Метод для подписки на новости
    public void Subscribe(INewsObserver observer)
    {
        if (!_subscribers.Contains(observer))
        {
            _subscribers.Add(observer);
        }
    }
}
```

```

        Console.WriteLine($"Новый подписчик добавлен к {_portalName}");
    }
}

// Метод для отписки от новостей
public void Unsubscribe(INewsObserver observer)
{
    if (_subscribers.Remove(observer))
    {
        Console.WriteLine($"Подписчик удален из {_portalName}");
    }
}

// Метод для уведомления всех подписчиков
public void NotifyObservers(string newsTitle, string newsContent)
{
    Console.WriteLine($"\\n{_portalName} публикует: {newsTitle}");
    Console.WriteLine($"Уведомление отправляется {_subscribers.Count} подписчикам...");

    foreach (var subscriber in _subscribers)
    {
        subscriber.Update(newsTitle, newsContent);
    }
}

// Метод для публикации новости с автоматическим уведомлением
public void PublishNews(string title, string content)
{
    Console.WriteLine($"\\nПУБЛИКАЦИЯ НОВОСТИ");
    NotifyObservers(title, content);
    Console.WriteLine($"ПУБЛИКАЦИЯ ЗАВЕРШЕНА\\n");
}
}

```

Класс NewsPortal инкапсулирует логику управления подписчиками: хранение списка подписчиков, добавление новых, удаление существующих и массовое уведомление при публикации новости. Важно отметить, что портал работает только с интерфейсом INewsObserver, не зная конкретных реализаций наблюдателей.

Теперь реализуем конкретных наблюдателей, которые будут реагировать на публикации новостей различными способами.

```

// Наблюдатель, получающий уведомления по электронной почте
public class EmailSubscriber : INewsObserver
{
    private string _subscriberEmail;
    private string _subscriberName;

    public EmailSubscriber(string name, string email)
    {
        _subscriberName = name;
        _subscriberEmail = email;
    }
}

```

```

        Console.WriteLine($"Создан email подписчик: {_subscriberName} ({_subscriberEmail})");
    }

    public void Update(string newsTitle, string newsContent)
    {
        Console.WriteLine($" Email для {_subscriberName} <{_subscriberEmail}>");
        Console.WriteLine($" Тема: {newsTitle}");
        Console.WriteLine($" Краткое содержание: {newsContent.Substring(0, Math.Min(50, newsContent.Length))}...");
    }
}

// Наблюдатель в виде мобильного приложения
public class MobileAppSubscriber : INewsObserver
{
    private string _appName;
    private string _deviceId;

    public MobileAppSubscriber(string appName, string deviceId)
    {
        _appName = appName;
        _deviceId = deviceId;
        Console.WriteLine($"Создан мобильный подписчик: {_appName} на устройстве {_deviceId}");
    }

    public void Update(string newsTitle, string newsContent)
    {
        Console.WriteLine($" Push-уведомление в {_appName} (устройство: {_deviceId})");
        Console.WriteLine($" {newsTitle}");
        Console.WriteLine($" Новость получена: {DateTime.Now:HH:mm:ss}");
        Console.WriteLine($" Конец уведомления \n");
    }
}

// Наблюдатель, сохраняющий новости в базу данных
public class DatabaseLogger : INewsObserver
{
    public void Update(string newsTitle, string newsContent)
    {
        Console.WriteLine($"Запись в базу данных:");
        Console.WriteLine($" Заголовок: {newsTitle}");
        Console.WriteLine($" Дата записи: {DateTime.Now:yyyy-MM-dd HH:mm:ss}");
        Console.WriteLine($" Статус: успешно сохранено");
        Console.WriteLine($" --- Конец уведомления ---\n");
    }
}

```

Каждый тип наблюдателя реализует метод Update по-своему, демонстрируя гибкость подхода. Email-подписчик формирует письмо, мобильное приложение создает push-уведомление, а система логирования сохраняет информацию в базу данных. При этом все они следуют одному интерфейсу, что позволяет новостному

порталу работать с любым типом подписчика, не зная деталей его реализации.

Выполнение этой программы демонстрирует ключевые преимущества паттерна "Наблюдатель". При публикации каждой новости все подписчики автоматически получают уведомления в соответствующем для них формате. Система позволяет динамически управлять подписками — добавлять и удалять подписчиков без остановки работы, причем один подписчик может отслеживать несколько издателей одновременно.

Основное преимущество такого подхода заключается в слабой связанности компонентов системы. Новостной портал не зависит от конкретных реализаций подписчиков — он работает только с интерфейсом `INewsObserver`.

Паттерн "Наблюдатель" естественным образом реализует принцип открытости/закрытости — система открыта для расширения (можно добавлять новые типы наблюдателей), но закрыта для модификации (не нужно изменять существующий код издателя при добавлении новых подписчиков). Это делает архитектуру устойчивой к изменениям бизнес-требований и упрощает поддержку системы в долгосрочной перспективе.

Таким образом, паттерн "Наблюдатель" предоставляет мощный механизм для организации взаимодействия между компонентами системы, позволяя создавать гибкие, расширяемые и легко поддерживаемые архитектуры, которые эффективно решают задачу распространения информации и реагирования на изменения состояния в сложных программных системах.

В ходе написания данного реферата был проведен комплексный анализ двух фундаментальных паттернов проектирования — "Фабричный метод" и "Наблюдатель" — в контексте языка программирования `C#`. Исследование подтвердило, что паттерны проектирования представляют собой не просто технические приемы, а целостные архитектурные решения, которые позволяют создавать гибкие, расширяемые и поддерживаемые